

一、论文浅析

1.几个相关的网格简化算法

--Vertex Decimation (顶点简化)

算法：选择一顶点，移除所有相邻面，把移除面后产生的洞重新三角化

特点：严格维护了模型的拓扑结构，且仅限于流形表面

--Vertex Clustering (顶点聚类)

算法：模型外面套个包围盒，再把盒子划分成很多块，每个块内的所有顶点聚合为一个顶点

特点：非常快，但质量不高

--Iterative Edge Contraction (迭代的边收缩)

算法：本论文的算法用到了这个思路，但选取待删除边的方式可能与先前算法不同。除此之外本论文的算法还有些独到之处，这在后面介绍。

2.算法简介

该算法可称为 *Iterative contraction of vertex pairs* (迭代的点对收缩)，一对点和边，这两者差在哪呢？

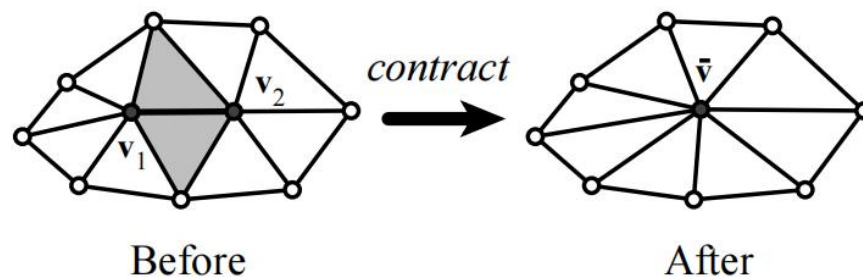


Figure 1: **Edge contraction.** The highlighted edge is contracted into a single point. The shaded triangles become degenerate and are removed during the contraction.

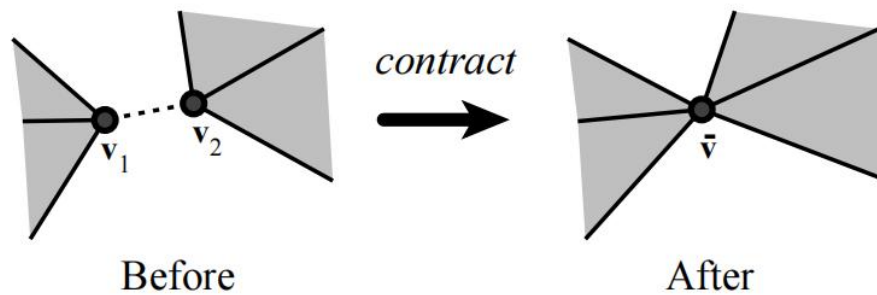


Figure 2: **Non-edge contraction.** When non-edge pairs are contracted, unconnected sections of the model are joined. The dashed line indicates the two vertices being contracted together.

差别就在于第二种情况。本算法支持把本不相连的部分连接起来（对于本次作业，这并不重要，而且目前未想到如何方便的找到距离很近的两个点，如果遍历所有点则时间复杂度过高。所以我为了方便没有实现这个功能，对不起 Michael Garland 老师，Paul S. Heckbert 老师，我就这么辜负了你们算法的独到之处！）



Figure 3: On the left is a regular grid 规则网格 of 100 closely spaced cubes. In the middle, an approximation built using only edge contractions demonstrates unacceptable fragmentation. On the right, the result of using more general pair contractions to achieve aggregation is an approximation much closer to the original.

作者也说了，他们更注重这种算法在渲染方面的应用，所以顶点简化等其他算法细心维持的拓扑结构（我也不太懂啦，抄一段 ai）

拓扑结构（Topology）是数学的一个分支，主要研究空间的性质，这些性质在连续变换下保持不变。换句话说，拓扑学关注的是空间中的对象在经过拉伸、弯曲等变形后仍然保持不变的性质，而不涉及对象的形状、大小或角度等具体特征。拓扑结构的概念也被广泛应用于计算机科学、网络科学和其他领域。

没有那么重要，反而是拓扑学不关注的形状、大小等整体外观比较重要。不相连处连接的聚合能力，很好的支持了整体外观上的优秀近似。

3. 算法内容

整体把握

算法要迭代地把一对点收缩为一个点，那我们得搞清楚以下几个问题：

- 哪些点对是有效点对(valid pairs)，可作为收缩的候选者？
- 在它们之中选哪对点？
- 收缩到哪个点？

(1) 哪些点对是有效点对(valid pairs)，可作为收缩的候选者？

We will say that a pair $(\mathbf{v}_1, \mathbf{v}_2)$ is a *valid* pair for contraction if either:

1. $(\mathbf{v}_1, \mathbf{v}_2)$ is an edge, *or*
2. $\|\mathbf{v}_1 - \mathbf{v}_2\| < t$, where t is a threshold parameter

是一条边，或者距离小于 t (t 的大小自己设置)。

(2) 在它们之中选哪对点？

当然是选对整体影响最小的一对点。而如何度量这个“影响”就是不同算法八仙过海，各显神通的事了。本文使用

$$\text{Cost} = \bar{\mathbf{v}}^T (\mathbf{Q}_1 + \mathbf{Q}_2) \bar{\mathbf{v}}$$

来度量影响，起名叫 cost 。 $\mathbf{Q}$ 是一个 4×4 对称矩阵，也是算法的核心。

(3) 收缩到哪个点？

这个其实在 (2) 中就解决了，因为计算 cost 就需要它 ($\bar{\mathbf{v}}$)。我们想要新产生的点使得 cost 最小，而我们的 cost 本质上是一个二次的式子……

$$\begin{aligned} \mathbf{v}^T \mathbf{Q} \mathbf{v} = & q_{11}x^2 + 2q_{12}xy + 2q_{13}xz + 2q_{14}x + q_{22}y^2 \\ & + 2q_{23}yz + 2q_{24}y + q_{33}z^2 + 2q_{34}z + q_{44} \end{aligned}$$

所以我们只要令 x, y, z 一阶偏导为零，如果能解出唯一的临界点，或者说唯一的极小值点（因为推导一下可知 Hessian 矩阵必是正定，这里不细说了），这个点就能使 cost 最小！（因为没有其他极值点了，所以极小值点应该能认定为最小值吧。【语气都变弱了！我数学真的不

好啊！】)

三个式子解三个未知数，若有唯一解，解出 $\bar{v}$ 就大功告成。这相当于论文中那个矩阵可逆的情况。如果有无穷解之类的情况论文中并未给出具体的处理方案，只说了在 v_1v_2 这条边上选最优点，还不行的话就在两端点和中点之间挑一个让 cost 更小的。为了省事，我直接挑了中点。

流程展示

1. Compute the $\mathbf{Q}$ matrices for all the initial vertices.
2. Select all valid pairs.
3. Compute the optimal contraction target $\bar{v}$ for each valid pair (v_1, v_2) . The error $\bar{v}^T(\mathbf{Q}_1 + \mathbf{Q}_2)\bar{v}$ of this target vertex becomes the *cost* of contracting that pair.
4. Place all the pairs in a heap keyed on cost with the minimum cost pair at the top.
5. Iteratively remove the pair (v_1, v_2) of least cost from the heap, contract this pair, and update the costs of all valid pairs involving v_1 .

The only remaining issue is how to compute the initial $\mathbf{Q}$ matrices from which the error metric Δ is constructed.

Q 的奥秘

$\mathbf{Q}$ 从哪个方面衡量了 cost ? 为什么仅靠加法就能实现 $\mathbf{Q}$ 的迭代? (这也是作者引以为豪的地方) 答案并不复杂。

可以通过 $\mathbf{Q}$ 计算某一点到所有关联平面距离的平方和。

论文的第 5 部分写的挺清楚的，我仅做一些补充说明。

注意：

1. v 是列向量，而且是四维列向量！ x, y, z 分量是对应的， w 分量恒为 1。
2. 在代码中用 `vecf4` 定义时，也默认为列向量

3. 代表平面的 p 向量，要求 a, b, c 平方和为 1
4. 在代码中，通过三角面上三个顶点的坐标，计算 p 向量（就是算平面方程）

在最开始，我们的每个顶点是带有一些关联平面的，包含这个顶点的所有三角面都是关联平面。代码中用 `faces_of_vertices` 表示。之前提到可以通过 Q 计算某一点到所有关联平面距离的平方和，具体来说， $v^T Q v$ 就等于点 v 到它所有关联平面距离的平方和。

当我们把两个点缩为一个新的点时，新点会把两个点的关联平面全部抢过来，占为己有。新点对应的新的 Q 变成了 $Q_1 + Q_2$ 。

$$\text{Cost} = \bar{v}^T (Q_1 + Q_2) \bar{v}$$

可以把这个式子拆开来理解一下， $v^T Q_1 v + v^T Q_2 v$

新点到这些关联平面距离的平方和，就是 cost 。

注意：

1. Q 里面的平面信息一直是初始的平面信息，新点的关联平面仍是初始模型上的平面，我们只是通过加法让新点获取两个旧点的关联平面，所以 cost 始终是相对于初始模型而言的。
2. 两个旧点的关联面通常是有交集的，直接 $Q_1 + Q_2$ 会导致少数平面重复计入，但作者认为这是可以接受的误差。

二、代码优化

1. 在 `mesh_simplification.h` 的 `Edge` 结构体中加入 `vecf3 vnew`，在计算初始 cost 时一并存入，这样后续循环的时候可以少进行一些计算。
2. 一开始采用了新增点的方法（比较便于理解），但后来变为在原有点上修改的方法，后者有两个好处：不用扩大各个数组的容量，节省了不少空间资源；更新 `faces_of_vertices` 时，可以少传入一个参数，也少一次遍历操作。
3. 对于堆的更新，一开始遍历了所有点对并全部重新计算 cost ，时间复杂度过高，运行过慢。经改进，对堆不断进行 `pop` 操作，若是无关边，直接把出来的元素放在 `temp` 数组中，若有关，计算后把新的点对和 cost 放入 `temp`，最后再把 `temp` 的元素重新 `push` 进堆中。

三、效果展示

